DOSSIER TECHNIQUE

SkyTrack

Plateforme de surveillance et d'analyse météorologique en architecture microservices

Projet microservices – Validation 1 : conception

Version 1.0 · novembre 2026

Membre	Microservice	Technologie
Ahmed Bouallegue	1 · User Service	.NET · PostgreSQL
Salim Khaldi	2 · Location Service	Spring Boot · PostgreSQL
Med Dhia Allagui	3 · Forecast Service	Spring Boot · MongoDB
Yasmine Fehri	4 · Alert & Notification Service	Spring Boot · MongoDB
Yassine Wesslati	5 · IoT Sensor Service	Spring Boot · InfluxDB
Abdallah Grissa	6 · Analytics & Preferences Service	Spring Boot · InfluxDB + MongoDB

Encadrant : ……

Établissement : ……

Dépôt : github.com/Ahmedbouallegue/skytrack_microservice_project-

Sommaire

1. Introduction 3
1.1 Contexte et problématique 3
1.2 Objectifs du projet 3
1.3 Périmètre de ce dossier 3
2. Analyse fonctionnelle 4
2.1 Acteurs 4
2.2 Fonctionnalités par acteur 4
3. Architecture 5
3.1 Principes de conception 5
3.2 Découpage en microservices 5
3.3 Architecture globale 6
3.4 Choix technologiques 7
4. Données 8
4.1 Une base de données par microservice 8
4.2 Modèle de données par microservice 8
4.3 Diagramme de classes en étoile 9
4.4 Exemple : document MongoDB du Forecast Service 10
5. Communication entre microservices 11
5.1 Appels entre services 11
5.2 Pourquoi des appels synchrones 11
5.3 Tolérance aux pannes 12
6. API REST 13
6.1 Conventions communes 13
6.2 Exemple : API du Forecast Service 13
7. Interface utilisateur 14
7.1 Deux espaces 14
7.2 Design system 14
7.3 Intégration dans Angular 14
8. Déploiement 15
8.1 Conteneurs Docker 15
8.2 Réseau 15
9. Organisation du projet 16
9.1 Dépôt Git 16
9.2 Méthode de travail 16
9.3 Outils 16
10. État d'avancement et prochaines étapes 17
10.1 Réalisé (validation 1) 17
10.2 Après validation 17
10.3 Conclusion 17

1. Introduction

1.1 Contexte et problématique

Les informations météorologiques utiles à une collectivité ou à un particulier sont aujourd'hui **dispersées** : les prévisions, les mesures relevées par les stations et les alertes officielles proviennent d'outils différents, qui ne communiquent pas entre eux.

- **Sources séparées** : prévisions, mesures des stations et alertes viennent de systèmes distincts.
- **Alertes génériques** : il n'existe pas de notification ciblée par ville ni par utilisateur.
- **Pas d'historique** : les mesures passées ne sont pas agrégées pour être analysées.

SkyTrack réunit ces informations dans une seule plateforme : prévisions par ville, mesures IoT en temps réel, alertes et notifications ciblées, historiques climatiques et préférences d'affichage.

1.2 Objectifs du projet

- Concevoir une application découpée en **6 microservices indépendants**, un par membre de l'équipe.
- Mettre en pratique trois familles de bases de données, chacune choisie selon la nature des données : **PostgreSQL** (relationnel), **MongoDB** (documents) et **InfluxDB** (séries temporelles).
- Faire communiquer les services uniquement par **API REST**, en échangeant des identifiants logiques.
- Fournir une interface **Angular** avec un espace public (front-office) et un espace d'administration (back-office).
- Rendre l'ensemble déployable avec **Docker** et tolérant aux pannes d'un service.

1.3 Périmètre de ce dossier

Ce dossier correspond à la **première validation**, consacrée à la conception. Il présente l'analyse fonctionnelle, le découpage en microservices, le modèle de données, les communications entre services, l'architecture technique, l'interface utilisateur et l'organisation de l'équipe. La logique métier de chaque service sera développée après validation de cette conception.

2. Analyse fonctionnelle

2.1 Acteurs

Acteur	Description	Espace
Utilisateur	Personne qui consulte la météo des villes qui l'intéressent et reçoit les alertes.	Front-office
Administrateur	Gère les comptes et les rôles, ainsi que le référentiel des villes et des régions.	Back-office
Prévisionniste	Publie les prévisions et déclenche les alertes climatiques.	Back-office
Station météo IoT	Acteur système, sans intervention humaine : envoie des mesures en continu.	API (IoT)

Tableau 1 – *Acteurs du système*

2.2 Fonctionnalités par acteur

Acteur	Fonctionnalité	Microservice
Utilisateur	Rechercher une ville et consulter ses prévisions (jour, heure par heure, 7 jours)	Forecast, Location
Utilisateur	Suivre des villes, choisir l'unité °C ou °F	Analytics & Preferences
Utilisateur	Recevoir les alertes et marquer une notification comme lue	Alert & Notification
Utilisateur	Consulter l'historique climatique d'une ville	Analytics & Preferences
Administrateur	Gérer les utilisateurs et leurs rôles	User
Administrateur	Gérer les villes et les régions	Location
Prévisionniste	Créer, modifier et supprimer des prévisions	Forecast
Prévisionniste	Déclencher et clôturer une alerte	Alert & Notification
Station IoT	Envoyer des mesures (température, humidité, pluie)	IoT Sensor

Tableau 2 – *Fonctionnalités et microservices concernés*

3. Architecture

3.1 Principes de conception

- **Un service = une capacité métier** : le découpage suit le métier, pas les tables ni les écrans. Aucune responsabilité n'est partagée entre deux services.
- **Une base de données par service** : aucun service ne lit la base d'un autre ; il passe par son API.
- **Relations logiques entre services** : pas de clé étrangère physique entre services, seulement des identifiants (`cityId`, `userId`).
- **Point d'entrée unique** : tout le trafic externe passe par l'API Gateway.
- **Tolérance aux pannes** : la panne d'un service ne doit pas provoquer d'erreurs en cascade.

3.2 Découpage en microservices

#	Microservice	Capacité métier	Technologie	Port
1	User Service	Comptes, rôles et authentification	ASP.NET Core (.NET)	8080
2	Location Service	Référentiel des villes et des régions	Spring Boot	8081
3	Forecast Service	Prévisions météo par ville	Spring Boot	8082
4	Alert & Notification	Alertes climatiques et notifications	Spring Boot	8083
5	IoT Sensor	Mesures en temps réel des stations	Spring Boot	8084
6	Analytics & Preferences	Historiques agrégés et préférences	Spring Boot	8085

Tableau 3 – *Découpage en microservices*

Le User Service est le seul développé en .NET. Comme tous les services communiquent en REST/JSON, les services Spring Boot l'appellent exactement comme les autres : le langage d'implémentation reste transparent.

3.3 Architecture globale

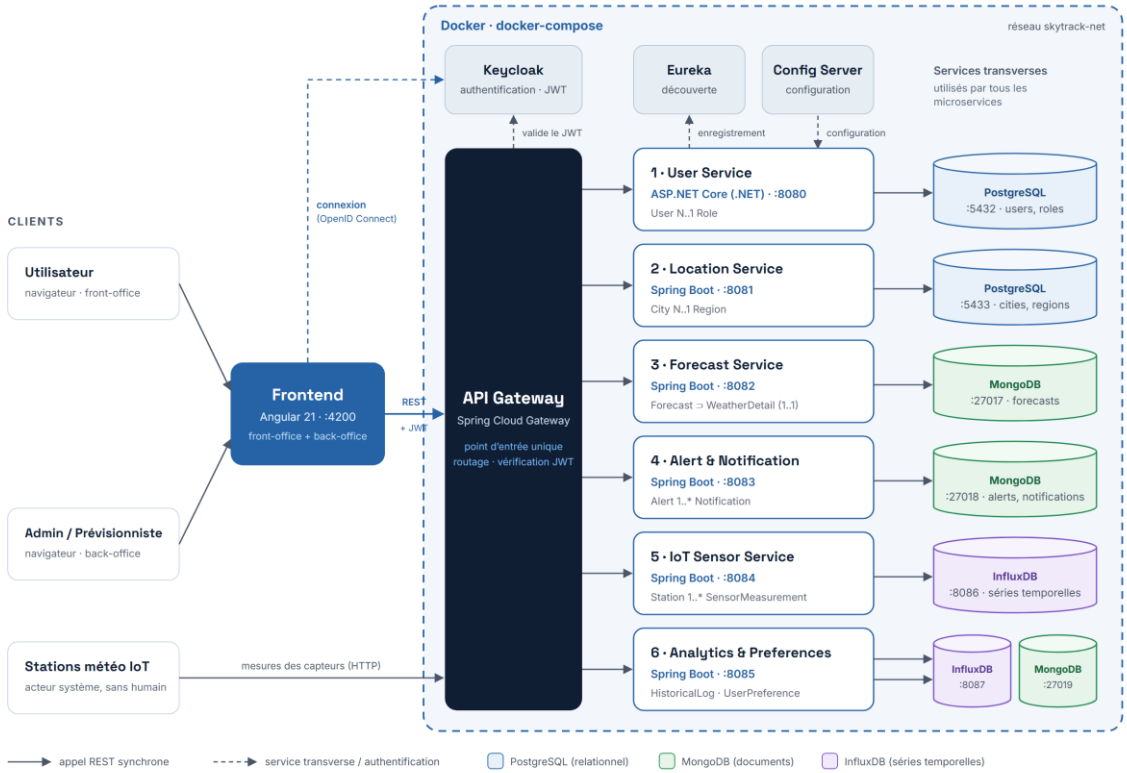


Figure 1 – *Architecture globale de SkyTrack*

- **Frontend Angular 21** (port 4200) : une seule application avec un front-office pour les utilisateurs et un back-office pour l'administration.
- **API Gateway** (Spring Cloud Gateway) : point d'entrée unique, routage vers les services et vérification du jeton JWT.
- **Keycloak** : authentification (OpenID Connect) et délivrance des jetons JWT, avec les rôles `ADMIN`, `FORECASTER` et `USER`.
- **Eureka** : chaque service s'y enregistre ; les autres le trouvent par son nom (ex. `location-service`), sans adresse codée en dur.
- **Config Server** : configuration centralisée de tous les services.
- **Docker** : chaque microservice, sa base et l'infrastructure tournent dans leur propre conteneur, lancés ensemble par docker-compose.

3.4 Choix technologiques

Couche	Technologie	Justification
Frontend	Angular 21, Bootstrap 5.3, Chart.js	Framework imposé ; Bootstrap pour un design cohérent et responsive ; Chart.js pour les graphiques.
Backend	Spring Boot 3.3 (Java 17)	Écosystème Spring Cloud complet : OpenFeign, Eureka, Gateway, Config, Resilience4j.
Backend	ASP.NET Core (.NET)	Montre l'interopérabilité : un service en C# intégré via REST/JSON.
Communication	REST/JSON, OpenFeign	Appels synchrones typés ; le client Feign résout le service via Eureka.
Résilience	Resilience4j	Timeout et circuit breaker sur chaque client Feign.
Sécurité	Keycloak (OIDC, JWT)	Authentification centralisée ; rôles transmis dans le jeton.
Bases	PostgreSQL, MongoDB, InfluxDB	Choisies selon la nature des données (voir chapitre 4).
Déploiement	Docker, docker-compose	Environnement identique pour toute l'équipe ; un conteneur par composant.

Tableau 4 – *Choix technologiques*

4. Données

4.1 Une base de données par microservice

Microservice	Base	Port	Pourquoi ce choix
User	PostgreSQL	5432	Données structurées et transactionnelles, relation User → Role.
Location	PostgreSQL	5433	Référentiel stable avec relation City → Region.
Forecast	MongoDB	27017	Une prévision et son détail forment un seul document, souvent corrigé.
Alert & Notification	MongoDB	27018	Le statut lu / non lu d'une notification est mis à jour.
IoT Sensor	InfluxDB	8086	Mesures horodatées écrites en continu, jamais modifiées.
Analytics & Preferences	InfluxDB + MongoDB	8087 / 27019	Moyennes horodatées (InfluxDB) et préférences modifiables (MongoDB).

Tableau 5 – *Bases de données par microservice*

> **Pourquoi pas InfluxDB pour les alertes ou les préférences ?** InfluxDB est optimisé pour l'ajout massif de points horodatés, mais gère mal la modification et ne connaît pas les relations. Une notification dont le statut change, ou une préférence que l'on lit et modifie, n'est pas une série temporelle : MongoDB est le bon choix.

4.2 Modèle de données par microservice

Microservice	Entités et attributs	Relation
User	**User** : id, username, email, password (haché), roleId **Role** : id, name	User N..1 Role (physique)
Location	**City** : id, name, latitude, longitude, regionId **Region** : id, name, country	City N..1 Region (physique)
Forecast	**Forecast** : id, cityId, date, summary, weatherDetail **WeatherDetail** : temperature, humidity, windSpeed, pressure	Composition 1..1 : WeatherDetail est embarqué dans Forecast
Alert & Notification	**Alert** : id, cityId, type, severity, createdAt **Notification** : id, alertId, userId, message, read	Alert 1..* Notification (alertId)
IoT Sensor	**Station** : stationId, name, cityId **SensorMeasurement** : timestamp, tag stationId, temperature, humidity, rain	Station 1..* SensorMeasurement (flux temporel)
Analytics & Preferences	**HistoricalLog** (InfluxDB) : cityId, period, avgTemperature, avgHumidity **UserPreference** (MongoDB) : id, userId, unit, alertsEnabled	Aucune relation entre les deux

Tableau 6 – *Entités, attributs et relations internes*

Les champs `cityId` et `userId` présents dans Forecast, Alert, Notification, Station, HistoricalLog et UserPreference sont des **références logiques** vers le Location Service et le User Service. Un `Long` côté Java correspond à un `long` côté C#.

4.3 Diagramme de classes en étoile

Le diagramme place **User** au centre et montre, pour chaque microservice, ses classes et leur lien avec l'utilisateur : relation physique (Role), référence logique par `userId` (Notification, UserPreference) ou simple consultation sans clé (Forecast, City, Station).

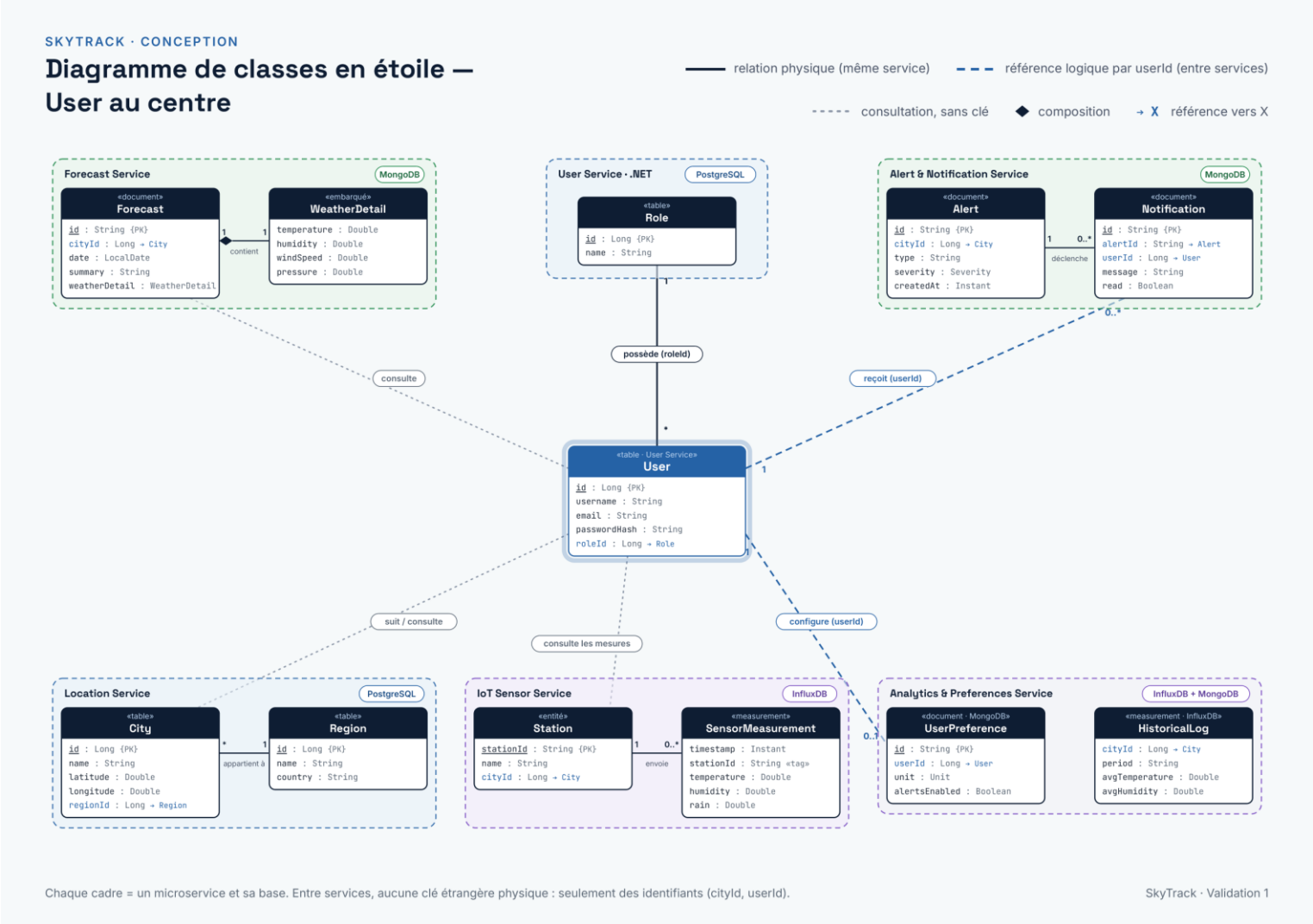


Figure 2 – *Diagramme de classes en étoile, User au centre*

4.4 Exemple : document MongoDB du Forecast Service

Le détail météo est un sous-document : il est lu et écrit en même temps que la prévision, sans jointure.

```
{
  "_id": "6730f1c2a4b5c6d7e8f90123",
  "cityId": 5,
  "date": "2026-11-10",
  "summary": "Ensoleillé",
  "weatherDetail": { "temperature": 21.5, "humidity": 60.0, "windSpeed": 12.0, "pressure": 1015.0
}
}
```

5. Communication entre microservices

5.1 Appels entre services

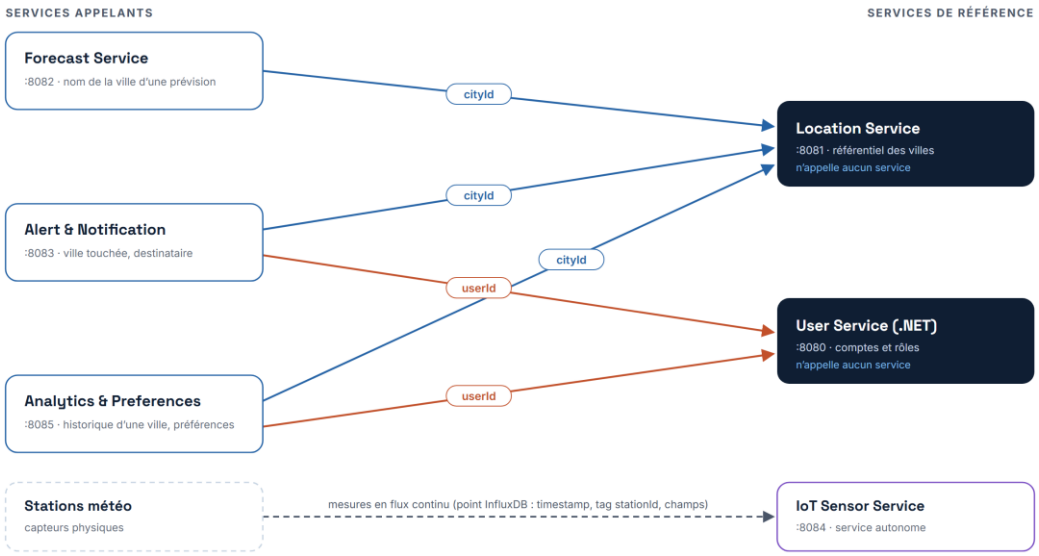


Figure 3 – *Appels entre microservices*

Service appelant	Service appelé	Identifiant	Objectif
Forecast	Location	cityId	Afficher le nom de la ville d'une prévision
Alert & Notification	Location	cityId	Connaître la ville touchée par une alerte
Alert & Notification	User	userId	Retrouver le destinataire d'une notification
Analytics & Preferences	Location	cityId	Ville d'un historique climatique
Analytics & Preferences	User	userId	Propriétaire des préférences

Tableau 7 – *Appels inter-services*

Location et User sont des **services de référence** : ils sont appelés mais n'appellent aucun autre service. L'IoT Sensor Service est **autonome** : il reçoit les mesures des stations et n'appelle personne.

5.2 Pourquoi des appels synchrones

Dans chaque cas, la réponse est nécessaire immédiatement pour répondre au client : à la création d'une prévision, il faut savoir tout de suite si le `cityId` existe ; à l'affichage, l'utilisateur attend le nom de la ville dans la même réponse. Un échange asynchrone par message broker ne conviendrait pas. Les appels utilisent **OpenFeign**, qui résout le service appelé par son nom Eureka.

5.3 Tolérance aux pannes

Chaque client Feign est protégé par un **timeout** (2 s) et un **circuit breaker** (Resilience4j). Si un service appelé ne répond pas, l'appelant passe en mode dégradé au lieu de propager l'erreur. Exemple du Forecast Service lorsque le Location Service est indisponible :

Opération	Comportement prévu
Lecture simple (GET /api/forecasts, /{id})	Aucun impact : n'appelle pas le Location Service.
Lecture enrichie (GET /{id}/details)	Mode dégradé : la prévision est renvoyée avec `cityName: null` (200).
Création (POST)	Refus avec 503 Service Unavailable : une ville non vérifiée n'est pas enregistrée.
Modification, suppression	Pas d'appel externe (le cityId n'est pas modifiable) : aucun impact.

Tableau 8 – *Mode dégradé du Forecast Service*

6. API REST

6.1 Conventions communes

- Chemin de base : `/api/<ressource au pluriel>` (ex. `/api/forecasts`, `/api/cities`).
- Format JSON ; les noms de champs sont ceux des entités.
- Codes HTTP : `200` lecture/modification, `201` création (avec en-tête `Location`), `204` suppression, `400` validation, `404` introuvable, `409` conflit, `503` dépendance indisponible.
- Toutes les routes sont exposées au frontend via l'API Gateway, qui vérifie le jeton JWT.

6.2 Exemple : API du Forecast Service

Méthode	URI	Description	Codes
GET	/api/forecasts	Liste ; filtres ?cityId= et ?date=	200
GET	/api/forecasts/{id}	Une prévision	200, 404
GET	/api/forecasts/{id}/details	Prévision + nom de la ville (appel Location)	200, 404
POST	/api/forecasts	Créer une prévision	201, 400, 404, 409, 503
PUT	/api/forecasts/{id}	Modifier une prévision	200, 400, 404
DELETE	/api/forecasts/{id}	Supprimer une prévision	204, 404

Tableau 9 – *API REST du Forecast Service*

Règles de validation : `cityId`, `date` et `weatherDetail` sont obligatoires ; `humidity` est comprise entre 0 et 100 ; `windSpeed` est positive ou nulle. Le `409` signale qu'une prévision existe déjà pour cette ville et cette date.

> Chaque responsable documente l'API de son service avec le même format dans le README de son dossier `Backend/<service>`.

7. Interface utilisateur

7.1 Deux espaces

Une maquette Bootstrap complète a été réalisée avant l'intégration Angular (`Template/skytrack-bootstrap`). Elle sépare deux espaces qui partagent le même design system ; la page de connexion redirige chaque utilisateur selon son rôle.

Espace	Acteurs	Pages
Front-office	Utilisateur	Accueil (recherche de ville, ville principale, vigilances, mes villes), page Ville (jour, station, heure par heure, 7 jours, historique), Mes alertes, Mon compte
Back-office	Administrateur, Prévisionniste	Tableau de bord (dont santé des services), Prévisions, Alertes, Stations IoT, Analyses, Villes et régions, Utilisateurs, Paramètres

Tableau 10 – *Pages de l'interface*

7.2 Design system

- **Jetons de design** : couleurs, rayons et ombres définis une seule fois en variables CSS ; aucune couleur codée en dur dans les pages.
- **Thème clair et sombre** : suit le réglage du système, l'utilisateur peut forcer l'un ou l'autre.
- **Statuts** : toujours une icône et un libellé (Critique, Sévère, Vigilance), jamais la couleur seule.
- **Graphiques** : une seule unité par graphique, trois séries au maximum avec une palette lisible par les daltoniens, une légende et une vue tableau.
- **Accessibilité et responsive** : navigation au clavier, libellés sur tous les champs ; sur téléphone, la navigation du front-office devient une barre d'onglets.

7.3 Intégration dans Angular

Le README de la maquette fixe des règles communes pour toute l'équipe : arborescence `core / shared / layouts / features/<service>`, correspondance page → route → composant → responsable, un seul service d'accès API par microservice, modèles TypeScript reprenant exactement les champs des entités, URL des API centralisées dans `environment.ts`, et liste de vérifications avant chaque livraison.

8. Déploiement

8.1 Conteneurs Docker

Chaque composant tourne dans son propre conteneur. Dans les conteneurs, chaque base écoute sur son port par défaut ; seul le port côté machine hôte change (notation hôte:conteneur).

Conteneur	Technologie	Ports
frontend	Angular 21	4200
api-gateway	Spring Cloud Gateway	à définir
eureka-server	Netflix Eureka	8761*
config-server	Spring Cloud Config	8888*
keycloak	Keycloak	8180:8080*
user-service + postgres-user	.NET + PostgreSQL	8080 · 5432:5432
location-service + postgres-location	Spring Boot + PostgreSQL	8081 · 5433:5432
forecast-service + mongo-forecast	Spring Boot + MongoDB	8082 · 27017:27017
alert-service + mongo-alert	Spring Boot + MongoDB	8083 · 27018:27017
iot-service + influx-iot	Spring Boot + InfluxDB	8084 · 8086:8086
analytics-service + influx-analytics + mongo-analytics	Spring Boot + InfluxDB + MongoDB	8085 · 8087:8086 · 27019:27017

Tableau 11 – *Conteneurs et ports*

* Ports par défaut proposés pour l'infrastructure, à confirmer par l'équipe. Keycloak est décalé sur 8180 pour ne pas entrer en conflit avec le User Service.

8.2 Réseau

Tous les conteneurs partagent le réseau Docker `skytrack-net` et se joignent par leur nom (ex. `mongo-forecast:27017`). Seuls le frontend et l'API Gateway doivent être exposés à l'extérieur ; les ports des bases ne sont publiés que pour le développement. Chaque service expose `/actuator/health`, utilisé pour superviser son état.

9. Organisation du projet

9.1 Dépôt Git

```
skytrack_microservice_project-/
├── Backend/              un projet par microservice (ex. forecast-service/)
├── Frontend/             application Angular
├── Template/             maquette Bootstrap (référence visuelle)
├── Documentation/
│   ├── Presentation/     présentation de la validation
│   └── Diagrams/         diagramme de classes, architecture globale
└── README.md
```

9.2 Méthode de travail

- **Une branche par membre** (ex. `dhia`) ; personne ne pousse directement sur `main`.
- **Pull request** vers `main` pour toute intégration, relue par un autre membre.
- **Commits réguliers** de chaque membre, avec des messages en français préfixés par le service concerné.
- **Suivi des tâches** dans GitHub Projects.
- Chaque service documente sa conception (responsabilités, modèle, API, interactions, pannes) dans son propre README.

9.3 Outils

Usage	Outils
Code source et revue	Git, GitHub, pull requests
Suivi des tâches	GitHub Projects
Build et exécution	Maven, .NET CLI, Angular CLI, Docker, docker-compose
Bases de données	PostgreSQL, MongoDB, InfluxDB
Conception	Diagramme de classes, maquette Bootstrap, diagrammes d'architecture

Tableau 12 – *Outils utilisés*

10. État d’avancement et prochaines étapes

10.1 Réalisé (validation 1)

- Sujet, acteurs et fonctionnalités définis.
- Découpage en six microservices, modèle de données et diagramme de classes.
- Communications inter-services et stratégie de tolérance aux pannes.
- Dépôt Git structuré, projets des microservices créés.
- Maquette de l’interface (front-office et back-office) et design system.
- Diagrammes d’architecture globale.

10.2 Après validation

1. Développer la logique métier et l’API REST de chaque service.
2. Mettre en place l’infrastructure : API Gateway, Eureka, Config Server et Keycloak.
3. Brancher les appels OpenFeign avec Resilience4j.
4. Intégrer l’interface dans Angular en suivant les règles communes de la maquette.
5. Écrire le docker-compose de l’ensemble et tester la plateforme complète.

10.3 Conclusion

La conception de SkyTrack répartit clairement les responsabilités entre six services autonomes, chacun avec la base de données adaptée à ses données. Les échanges se limitent à des identifiants, les pannes sont contenues, et des conventions communes (API, interface, Git) garantissent qu’un code écrit par six personnes reste cohérent une fois intégré.